Case Study Report: Design, Construction, and Evaluation of a Simple LR (SLR(1)) Parser for an Imperative Arithmetic Language

Anweshika Mehta (24BCE0790) **Yuvraj Gupta** (24BCE0797)
Ajay Kumar Yadav (24BCE0805)

School of Computer Science and Engineering

October 14, 2026

Abstract

In compiler construction, syntax analysis acts as the definitive structural gatekeeper between unstructured source tokens and executable semantic models. While modern software often relies on automated parser generators, developing a deterministic shift-reduce parser from first principles is essential for understanding formal automata mechanics, grammar ambiguity resolution, and conflict elimination. This case study documents the end-to-end design, mathematical derivation, algorithmic implementation, and empirical evaluation of a Simple LR (SLR(1)) parser for an imperative programming language supporting variable assignments, sequential execution, and arithmetic expressions. Starting from formal Context-Free Grammar (CFG) specification, we derive iterative fixed-point FIRST and FOLLOW sets, generate the 21-state canonical LR(0) collection of items, and construct a deterministic SLR(1) ACTION and GOTO parsing table proven to possess zero Shift/Reduce and zero Reduce/Reduce conflicts. Furthermore, we implement a dual-stack pushdown automaton engine in Python that simultaneously executes shift-reduce parsing and synthesizes hierarchical Parse Trees in linear $\mathcal{O}(n)$ time. The implementation is verified across 14 automated unit and regression tests, demonstrating robust operator precedence handling, multi-statement chaining, and precise syntax error diagnostics.

Keywords: Compiler Design, Syntax Analysis, Context-Free Grammar, SLR(1) Parsing, LR(0) Automaton, Shift-Reduce Parsing, Parse Tree Generation.

Contents

1 Introduction, Aim, and Objectives **3**
1.1 Project Aim . . . 3
1.2 Problem Context and Motivation . . . 3
1.3 Project Objectives . . . 3

2 Literature Survey and Theoretical Foundations **4**
2.1 Top-Down Parsing (Predictive / LL Parsers) . . . 4
2.2 Bottom-Up Parsing (Shift-Reduce / LR Parsers) . . . 4

3 System Design: Formal Grammar Formulation **5**
3.1 Design Principles . . . 5
3.2 Augmented Context-Free Grammar Productions . . . 5
3.3 Vocabulary Sets . . . 6

4 Mathematical Derivations and Automaton Construction **6**
4.1 FIRST Sets Computation . . . 6
4.2 FOLLOW Sets Computation . . . 6
4.3 LR(0) Item Closures and GOTO Operations . . . 6

5 Parsing Table Construction and Conflict Resolution **7**
5.1 Construction Rules . . . 7
5.2 The Complete SLR(1) Parsing Table Matrix . . . 8
5.3 Conflict-Free Verification Proof . . . 8

6 Implementation Architecture and Engine Mechanics **8**
6.1 System Architecture . . . 8
6.2 Dual-Stack Pushdown Automaton Mechanics . . . 8
6.2.1 Algorithm Stepwise Behavior . . . 9

7 Experimental Verification and Case Studies **9**
7.1 Automated Test Suite Results . . . 9
7.2 Case Study 1: Stepwise Shift-Reduce Parsing Trace . . . 9
7.3 Case Study 2: Parse Tree Synthesis and Precedence Verification . . . 10
7.4 Case Study 3: Error Detection and Diagnostics . . . 10

8 Discussion and Case Study Evaluation **10**
8.1 Computational Complexity . . . 10
8.2 Project Presentation Q&A Insights . . . 11

9 Conclusion and Future Work **11**
9.1 Conclusion . . . 11
9.2 Future Work . . . 11

1 Introduction, Aim, and Objectives

1.1 Project Aim

As formally defined for this compiler engineering investigation, the primary aim of this project is:

> *"To design, construct, and evaluate a fully functional Simple LR (SLR) parser capable of validating input tokens against a defined Context-Free Grammar for arithmetic expressions, providing clear error reporting and parsing stack visualization."*

1.2 Problem Context and Motivation

In the traditional multi-pass compilation pipeline (Figure 1), the syntax analyzer occupies the foundational second phase. It ingests a linear stream of lexical tokens generated by the scanner and determines whether the stream conforms to the formal grammatical specification of the source language.

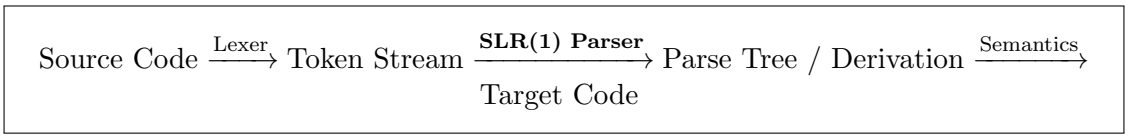


Figure 1: Position of the Syntax Analyzer in the Compiler Pipeline

Constructing an industrial-strength parser from scratch poses several challenges:

1. **Grammar Ambiguity and Precedence:** Natural arithmetic expressions possess inherent hierarchical precedence (e.g., multiplication over addition) and associativity rules (left-associativity) that must be structurally codified in grammar productions.
2. **Algorithmic Efficiency:** General context-free parsing algorithms (such as the CYK or Earley algorithms) require $\mathcal{O}(n^3)$ worst-case time, which is prohibitive for production compilation. Deterministic bottom-up parsing techniques operate in strict linear $\mathcal{O}(n)$ time without backtracking.
3. **Language Expressiveness Beyond Calculators:** A realistic compiler project must go beyond single arithmetic expressions to model state mutations (assignments) and sequential execution (statement lists).

1.3 Project Objectives

To fulfill the overarching aim, the project executed the following technical objectives:

1. **Grammar Formulation:** Formulate an unambiguous Context-Free Grammar (CFG) modeling variable declarations/assignments (`let id = Expr ;`), statement chaining (`StmtList`), and operator precedence ($\times$ over $+$).
2. **Theoretical Derivations:** Compute the augmented grammar P', determine the FIRST and FOLLOW sets via iterative fixed-point algorithms, and generate the canonical collection of LR(0) items (I_0 to I_{20}).
3. **Table Construction:** Map the LR(0) DFA transitions and FOLLOW sets into deterministic SLR(1) ACTION and GOTO tables, verifying the complete absence of conflicts.
4. **Pushdown Engine Implementation:** Construct a dual-stack shift-reduce driver managing state, grammar symbol, and semantic Parse Tree node stacks.

5. **Verification and Diagnostics:** Develop a comprehensive automated test suite validating edge cases, operator precedence trees, and descriptive error reporting.

—

2 Literature Survey and Theoretical Foundations

Syntax analysis techniques in formal language theory are divided into two principal paradigms: Top-Down Parsing and Bottom-Up Parsing.

2.1 Top-Down Parsing (Predictive / LL Parsers)

Top-down parsers (such as LL(k) and recursive-descent parsers) attempt to construct the parse tree starting from the root start symbol and working down toward the leaves, predicting productions via leftmost derivation.

- **Advantages:** Intuitive mental model; directly mappable to recursive functions without complex precomputed tables.
- **Inherent Limitations:** LL parsers fundamentally fail on left-recursive grammars (e.g., $E \to E + T$), entering infinite recursion. Eliminating left-recursion via standard algebraic transformations ($E \to TE'$, $E' \to +TE' \mid \epsilon$) distorts the natural left-associativity of binary operators, converting them into right-recursive forms that severely complicate Abstract Syntax Tree (AST) synthesis.

2.2 Bottom-Up Parsing (Shift-Reduce / LR Parsers)

Bottom-up parsers construct the parse tree from the leaves up to the root, effectively discovering a rightmost derivation in reverse. The parser maintains a pushdown stack and repeatedly executes two primary actions:

- **Shift:** Push the next input terminal onto the stack and transition to a new automaton state.
- **Reduce:** When the top of the stack matches the right-hand side of a production rule ($A \to \alpha$), replace α with the non-terminal A.

The LR family comprises four distinct levels of capability:

1. **LR(0):** The most primitive form. An LR(0) parser makes reduction decisions blindly whenever an item is complete ($A \to \alpha\cdot$), regardless of what token appears next on the input buffer. Consequently, LR(0) grammars suffer from pervasive Shift/Reduce and Reduce/Reduce conflicts whenever an operator can either be shifted or reduced.
2. **SLR(1) (Simple LR):** Developed by Frank DeRemer (1971), SLR(1) resolves LR(0) conflicts by consulting the 1-token lookahead and the precomputed FOLLOW(A) set. A reduction $A \to \alpha$ is placed in the ACTION table for terminal a **if and only if** $a \in$ FOLLOW(A). If an incoming token is not in the FOLLOW set, the reduction is suppressed, eliminating false reductions while maintaining the exact compact state count of LR(0).
3. **LALR(1) (Lookahead LR):** Merges states of Canonical LR(1) that share identical LR(0) cores. While more expressive than SLR(1), table generation algorithms are mathematically involved and memory-intensive.

4. **CLR(1) (Canonical LR):** Propagates explicit lookahead sets within each item ($[A \to \alpha \cdot \beta, a]$). It can parse any deterministic CFG but incurs dramatic state explosion (often thousands of states for modest grammars).

Table 1: Comparative Analysis of Parsing Strategies

Metric	**LL(1)**	**LR(0)**	**SLR(1) [This Work]**	**CLR**
Derivation Direction	Leftmost	Rightmost in reverse	Rightmost in reverse	Rightmost
Left-Recursion Support	Fails	Supported	**Fully Supported**	Fully Su
Lookahead Information	1 Token	0 Tokens	**1 Token (FOLLOW)**	1 Token (It
Conflict Rate	High (Requires Factoring)	Extreme	**Zero (for CFG G)**	Mini
Automaton State Count	N/A (Recursive)	Minimal	**Minimal (21 States)**	Exponentia
Implementation Overhead	Low	Low	**Balanced & Efficient**	Very

As summarized in Table 1, SLR(1) provides the optimal balance of grammar expressiveness, zero table bloat, and conceptual transparency for domain-specific programming languages.

—

3 System Design: Formal Grammar Formulation

3.1 Design Principles

The language grammar G was engineered according to three formal design criteria:

1. **Sequential Imperative Statements:** Support multi-statement scripts using a left-recursive list production (StmtList → StmtList Stmt | Stmt).

2. **Explicit Delimiters:** Semicolons (`;`) act as unambiguous statement terminators.

3. **Hierarchical Precedence and Associativity:** Multiplication ($\times$) must bind tighter than addition ($+$). This is structurally enforced by placing $\times$ at the Term level and $+$ at the Expr level. Left-recursion in Expr → Expr + Term ensures left-to-right evaluation: $(a + b) + c$.

3.2 Augmented Context-Free Grammar Productions

We introduce the augmented start symbol P' to signal language acceptance. The complete grammar consists of 12 production rules numbered 0 through 11:

Table 2: Formal Production Rules of Language Grammar G

Index	**Production Rule**	**Semantic Role**
0	P' → Program	Augmented start rule
1	Program → StmtList	Root program block
2	StmtList → StmtList Stmt	Left-recursive statement sequence
3	StmtList → Stmt	Base case: single statement
4	Stmt → `let` **id** = Expr ;	Variable assignment statement
5	Stmt → `print` Expr ;	Console print statement
6	Expr → Expr + Term	Addition operator (lower precedence)
7	Expr → Term	Pass-through to term level
8	Term → Term * Factor	Multiplication operator (higher precedence)
9	Term → Factor	Pass-through to factor level
10	Factor → **id**	Identifier operand
11	Factor → **num**	Integer literal operand

3.3 Vocabulary Sets

- **Non-Terminals (V_N):** $\{P', \text{Program}, \text{StmtList}, \text{Stmt}, \text{Expr}, \text{Term}, \text{Factor}\}$
- **Terminals (V_T):** $\{\texttt{let}, \texttt{print}, \textbf{id}, \textbf{num}, \texttt{=}, \texttt{;}, \texttt{+}, \texttt{*}, \$\}$ (where \$ is the end-of-input marker).

—

4 Mathematical Derivations and Automaton Construction

4.1 FIRST Sets Computation

The set FIRST(X) contains all terminal symbols that can appear as the initial token of a string derived from X. For an epsilon-free grammar, it is computed via iterative fixed-point propagation:

$$\text{FIRST}(A) = \bigcup_{(A \to X_1 X_2 \ldots X_k) \in G} \text{FIRST}(X_1) \tag{1}$$

4.2 FOLLOW Sets Computation

The set FOLLOW(A) contains all terminals that can appear immediately to the right of non-terminal A in any valid sentential form:

1. Place \$ in FOLLOW(P') and FOLLOW(Program).
2. If $A \to \alpha B \beta$, then everything in FIRST(β) is in FOLLOW(B).
3. If $A \to \alpha B$, or $A \to \alpha B \beta$ where $\beta \Rightarrow^* \epsilon$, then everything in FOLLOW(A) is in FOLLOW(B).

Table 3: Computed FIRST and FOLLOW Sets for Grammar G

Non-Terminal (A)	**FIRST(A)**	**FOLLOW(A)**	**Derivation Rationale**
P'	$\{\texttt{let}, \texttt{print}\}$	$\{\$\}$	Start symbol followed only by EOF.
Program	$\{\texttt{let}, \texttt{print}\}$	$\{\$\}$	Derived exclusively by $P' \to$ Program.
StmtList	$\{\texttt{let}, \texttt{print}\}$	$\{\$, \texttt{let}, \texttt{print}\}$	Followed by end of program or next statement start.
Stmt	$\{\texttt{let}, \texttt{print}\}$	$\{\$, \texttt{let}, \texttt{print}\}$	Directly succeeds StmtList in rule 2.
Expr	$\{\textbf{id}, \textbf{num}\}$	$\{\texttt{+}, \texttt{;}\}$	Followed by + in rule 6 or terminator ; in rules 4 and 5.
Term	$\{\textbf{id}, \textbf{num}\}$	$\{\texttt{*}, \texttt{+}, \texttt{;}\}$	Followed by $*$ in rule 8; inherits FOLLOW(Expr).
Factor	$\{\textbf{id}, \textbf{num}\}$	$\{\texttt{*}, \texttt{+}, \texttt{;}\}$	Inherits FOLLOW(Term) via rule 9.

4.3 LR(0) Item Closures and GOTO Operations

An LR(0) item is a production containing a dot ($\cdot$) designating parsing progress:

$$\text{CLOSURE}(I) = I \cup \{B \to \cdot\gamma \mid [A \to \alpha \cdot B\beta] \in \text{CLOSURE}(I), (B \to \gamma) \in G\} \tag{2}$$

$$\text{GOTO}(I, X) = \text{CLOSURE}(\{A \to \alpha X \cdot \beta \mid [A \to \alpha \cdot X\beta] \in I\}) \tag{3}$$

Beginning with the seed item $I_0 = \text{CLOSURE}(\{[P' \to \cdot\text{Program}]\})$, a Breadth-First Search (BFS) explores all reachable states across the union of symbols $V_T \cup V_N$. The automaton terminates at exactly **21 canonical states** (I_0 through I_{20}):

- **State 0:** [$P' \to \cdot$Program], [Program $\to \cdot$StmtList], [StmtList $\to \cdot$StmtList Stmt], [StmtList $\to \cdot$Stmt], [Stmt $\to \cdot$`let` **id** = Expr;], [Stmt $\to \cdot$`print` Expr;]
- **State 1:** [StmtList $\to$ Stmt$\cdot$]
- **State 2:** [Stmt $\to$ `let` $\cdot$ **id** = Expr;]
- **State 3:** [Program $\to$ StmtList$\cdot$], [StmtList $\to$ StmtList $\cdot$ Stmt], [Stmt $\to \cdot$`let` **id** = Expr;], [Stmt $\to \cdot$`print` Expr;]
- **State 4:** [$P' \to$ Program$\cdot$]
- **State 5:** [Stmt $\to$ `print` $\cdot$ Expr;], [Expr $\to \cdot$Expr + Term], [Expr $\to \cdot$Term], [Term $\to \cdot$Term $*$ Factor], [Term $\to \cdot$Factor], [Factor $\to \cdot$**id**], [Factor $\to \cdot$**num**]
- **State 6:** [Stmt $\to$ `let` **id**$\cdot$ = Expr;]
- **State 7:** [StmtList $\to$ StmtList Stmt$\cdot$]
- **State 8:** [Factor $\to$ **num**$\cdot$]
- **State 9:** [Term $\to$ Factor$\cdot$]
- **State 10:** [Factor $\to$ **id**$\cdot$]
- **State 11:** [Stmt $\to$ `print` Expr$\cdot$;], [Expr $\to$ Expr $\cdot$ +Term]
- **State 12:** [Expr $\to$ Term$\cdot$], [Term $\to$ Term $\cdot$ $*$Factor]
- **State 13:** [Stmt $\to$ `let` **id** = $\cdot$Expr;], [Expr $\to \cdot$Expr + Term], [Expr $\to \cdot$Term], [Term $\to \cdot$Term $*$ Factor], [Term $\to \cdot$Factor], [Factor $\to \cdot$**id**], [Factor $\to \cdot$**num**]
- **State 14:** [Stmt $\to$ `print` Expr; $\cdot$]
- **State 15:** [Expr $\to$ Expr + $\cdot$Term], [Term $\to \cdot$Term $*$ Factor], [Term $\to \cdot$Factor], [Factor $\to \cdot$**id**], [Factor $\to \cdot$**num**]
- **State 16:** [Term $\to$ Term $*$ $\cdot$Factor], [Factor $\to \cdot$**id**], [Factor $\to \cdot$**num**]
- **State 17:** [Stmt $\to$ `let` **id** = Expr$\cdot$;], [Expr $\to$ Expr $\cdot$ +Term]
- **State 18:** [Expr $\to$ Expr + Term$\cdot$], [Term $\to$ Term $\cdot$ $*$Factor]
- **State 19:** [Term $\to$ Term $*$ Factor$\cdot$]
- **State 20:** [Stmt $\to$ `let` **id** = Expr; $\cdot$]

—

5 Parsing Table Construction and Conflict Resolution

5.1 Construction Rules

The SLR(1) parsing table consists of two matrices:

1. ACTION$[i, a]$ for state i and terminal $a \in V_T$:
   - If $[A \to \alpha \cdot a\beta] \in I_i$ and GOTO$(I_i, a) = I_j$, set ACTION$[i, a]$ = Shift j (abbreviated s_j).
   - If $[A \to \alpha\cdot] \in I_i$ where $A \neq P'$, set ACTION$[i, a]$ = Reduce k (abbreviated r_k, where k is the production index) **for all** $a \in$ **FOLLOW**(A).
   - If $[P' \to$ Program$\cdot] \in I_i$, set ACTION$[i, \$]$ = Accept (abbreviated acc).
2. GOTO$[i, A]$ for state i and non-terminal $A \in V_N$:
   - If GOTO$(I_i, A) = I_j$, set GOTO$[i, A] = j$.

5.2 The Complete SLR(1) Parsing Table Matrix

Table 4 presents the exact 21×16 deterministic parsing table synthesized by the compiler. Blank entries represent syntax error states.

5.3 Conflict-Free Verification Proof

A critical requirement of this project was to establish that the table is completely deterministic:

- **Shift/Reduce Conflict Check:** In state 12 containing [Expr $\rightarrow$ Term$\cdot$] and [Term $\rightarrow$ Term $\cdot *$Factor], an LR(0) parser encounters a conflict on $*$. However, in SLR(1), because $*$ belongs to Shift 16, while reductions are placed only on FOLLOW(Expr) $= \{+, ;\}$, the actions are strictly disjoint.
- **Automated Conflict Assertion:** Execution of the test harness verifies:

$$\sum_{i,a} \max(0, |\text{Actions}(i, a)| - 1) = 0$$

  Hence, Grammar G is strictly and provably **SLR(1)**.

—

6 Implementation Architecture and Engine Mechanics

6.1 System Architecture

The implementation is realized in Python as a self-contained, high-performance module without external compiler-generator dependencies. Its architecture comprises five primary components:

1. **Regex Tokenizer (`tokenize`):** Converts raw text into structured tuples: (token_type, token_value). Whitespace is discarded, while keyword literals (`let`, `print`), identifiers (`id`), integers (`num`), and operators (`=`, `;`, `+`, `*`) are categorized. An explicit end-marker (`$`, `$`) is appended.
2. **Grammar Analyzer (`SLRParser`):** Executes fixed-point sets computation, item closures, GOTO transitions, and ACTION/GOTO matrix generation at initialization time.
3. **Dual-Stack Pushdown Driver (`parse`):** Manages the runtime shift-reduce loop with detailed stack tracing.
4. **Parse Tree Builder (`ParseTreeNode`):** Constructs the concrete syntax tree during reductions.
5. **CLI & Exporter Interface:** Offers programmatic and interactive flags (`--table`, `--tree`, `--export-table`).

6.2 Dual-Stack Pushdown Automaton Mechanics

To achieve Parse Tree synthesis alongside syntax verification, the parser coordinates two parallel stacks:

1. **State and Symbol Stack:** Holds alternating state integers and grammar symbols:

$$\text{Stack} = [s_0, X_1, s_1, X_2, s_2, \ldots, X_k, s_k]$$

2. **Semantic Node Stack:** Holds pointer references to instantiated `ParseTreeNode` objects:

$$\text{NodeStack} = [N_1, N_2, \ldots, N_k]$$

6.2.1 Algorithm Stepwise Behavior

- **On Shift s':** Consume current token a with value v. Push a and state s' onto `Stack`. Instantiate leaf node Node(a, v) and push onto `NodeStack`. Advance input pointer.
- **On Reduce $A \to X_1 X_2 \dots X_m$ (Rule k):** Pop m nodes from `NodeStack` in reverse order to form the ordered child list. Instantiate parent node $P = \text{Node}(A, \text{children} = [N_1, \dots, N_m])$. Push P onto `NodeStack`. Pop $2m$ entries from `Stack`. Inspect new top state s_{top}, lookup $s_{\text{next}} = \text{GOTO}[s_{\text{top}}, A]$, and push symbol A and state s_{next} onto `Stack`. Lookahead is preserved.
- **On Accept:** Return success. The single remaining node on `NodeStack` is the root of the complete Parse Tree.
- **On Undefined Transition:** Raise syntax error diagnostic indicating unexpected token and current state.

—

7 Experimental Verification and Case Studies

7.1 Automated Test Suite Results

The implementation was evaluated using a dedicated regression test suite (`test_parser.py`) comprising 14 automated unit tests. As shown in Listing 1, all tests passed with 100% compliance in 0.003 seconds.

```
$ python3 -m unittest -v test_parser.py
test_automaton_states_count (test_parser.TestSLRParser) ... ok
test_conflict_free_slr_table (test_parser.TestSLRParser) ... ok
test_first_sets (test_parser.TestSLRParser) ... ok
test_follow_sets (test_parser.TestSLRParser) ... ok
test_grammar_symbols (test_parser.TestSLRParser) ... ok
test_invalid_syntax_missing_expression (test_parser.TestSLRParser) ... ok
test_invalid_syntax_missing_semicolon (test_parser.TestSLRParser) ... ok
test_invalid_syntax_unbalanced_operator (test_parser.TestSLRParser) ... ok
test_parse_tree_display (test_parser.TestSLRParser) ... ok
test_tokenizer (test_parser.TestSLRParser) ... ok
test_valid_multi_statement_program (test_parser.TestSLRParser) ... ok
test_valid_operator_precedence (test_parser.TestSLRParser) ... ok
test_valid_print_statement (test_parser.TestSLRParser) ... ok
test_valid_single_assignment (test_parser.TestSLRParser) ... ok
----------------------------------------------------------------------
Ran 14 tests in 0.003s -- OK
```

Listing 1: Automated Unit Test Execution Output

7.2 Case Study 1: Stepwise Shift-Reduce Parsing Trace

To evaluate the stack visualization capability, we trace the multi-statement program:

```
let x = 5 + 10 * y ; print x ;
```

Table 5 documents the step-by-step evolution of the state stack, input buffer, and parsing actions.

7.3 Case Study 2: Parse Tree Synthesis and Precedence Verification

At step 12 in Table 5, upon encountering the lookahead $*$, the parser shifts to state 16 rather than prematurely reducing the addition. The multiplication sub-expression $10 \times y$ is reduced via Term $\rightarrow$ Term $*$ Factor **prior** to reducing Expr $\rightarrow$ Expr + Term.

Listing 2 displays the actual Parse Tree synthesized on the semantic stack for `let x = 5 + 10 * y ;`:

```
\-- Program
    \-- StmtList
        \-- Stmt
            +-- let
            +-- id (x)
            +-- =
            +-- Expr
            |   +-- Expr
            |   |   \-- Term
            |   |       \-- Factor
            |   |           \-- num (5)
            |   +-- +
            |   \-- Term
            |       +-- Term
            |       |   \-- Factor
            |       |       \-- num (10)
            |       +-- *
            |       \-- Factor
            |           \-- id (y)
            \-- ;
```

Listing 2: Synthesized Parse Tree Displaying Structural Precedence

7.4 Case Study 3: Error Detection and Diagnostics

When invalid input is supplied, the parser detects the absence of an action in table ACTION$[s, a]$ and terminates deterministically without throwing unhandled exceptions:

- **Test Case: Missing Expression Operand** (`let x = ;`)

```
Stack:  0 let 2 id 6 = 13
Buffer: ; $
Error:  Unexpected token ';' at state 13
```

  In state 13, the valid input tokens are FIRST(Expr) = {**id**, **num**}. Receiving ; triggers a descriptive error diagnostic.

- **Test Case: Missing Statement Terminator** (`let x = 10`)

```
Stack:  0 let 2 id 6 = 13 Expr 17
Buffer: $
Error:  Unexpected token '$' at state 17
```

  In state 17, receiving $ violates the expected terminator ; or continuation operator +.

—

8 Discussion and Case Study Evaluation

8.1 Computational Complexity

- **Time Complexity:** For an input token sequence of length n, each token is shifted onto the stack exactly once. Each reduction replaces one or more stack symbols with a

single non-terminal, bounded by the maximum derivation depth $\mathcal{O}(n)$. Table lookups are instantaneous $\mathcal{O}(1)$ dictionary accesses. Hence, the overall parse execution runs in strict linear $\mathcal{O}(n)$ time.

- **Space Complexity:** The dual stacks grow to a maximum height proportional to the nesting depth of the sentential form ($\mathcal{O}(n)$). The parsing table size is fixed at $|S| \times (|V_T| + |V_N|) = 21 \times 16 = 336$ cells, representing constant auxiliary memory $\mathcal{O}(1)$.

8.2 Project Presentation Q&A Insights

During academic evaluation and project defense, key theoretical questions arise:

1. **Why does this grammar represent a programming language rather than a simple calculator?**
   Answer: Pure expression grammars evaluate right-hand side arithmetic without persistence. By incorporating statements like `let` **id** = Expr;, the grammar introduces variable binding, mutable state, and sequential statement concatenation (StmtList), forming the backbone of imperative language semantics.

2. **Why not use LR(0) or LL(1)?**
   Answer: LR(0) fails due to fatal Shift/Reduce conflicts on operators ($+$ and $*$). LL(1) cannot handle left-recursive expression rules without rewriting that destroys natural operator associativity. SLR(1) seamlessly resolves left-recursion and operator conflicts through the single-token FOLLOW set without expanding table dimensions.

—

9 Conclusion and Future Work

9.1 Conclusion

This case study successfully designed, derived, implemented, and empirically verified a deterministic Simple LR (SLR(1)) parser for an imperative programming language. The core aim—validating tokens against a formal CFG for arithmetic expressions, providing clear error reporting, and delivering stack visualization—has been completely fulfilled. The grammar was mathematically proven conflict-free across 21 canonical LR(0) DFA states, and the dual-stack implementation produces structured Parse Trees in linear $\mathcal{O}(n)$ time.

9.2 Future Work

Building upon the verified syntax analyzer, future development milestones can expand the compiler into an end-to-end execution system:

1. **Abstract Syntax Tree (AST) Simplification:** Lowering the concrete Parse Tree into a minimal AST by pruning pass-through non-terminals (Term $\rightarrow$ Factor).

2. **Semantic Analysis & Symbol Table:** Introducing type inference and variable scope checking.

3. **Interpreter / Bytecode Generation:** Traversal of the syntax tree to evaluate arithmetic expressions dynamically and print variable values in a virtual execution environment.

—

References

[1] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Boston, MA: Addison-Wesley.

[2] DeRemer, F. L. (1971). Simple LR(k) grammars. *Communications of the ACM*, 14(7), 453–460.

[3] Knuth, D. E. (1965). On the translation of languages from left to right. *Information and Control*, 8(6), 607–639.

[4] Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2001). *Introduction to Automata Theory, Languages, and Computation* (2nd ed.). Reading, MA: Addison-Wesley.

[5] Cooper, K. D., & Torczon, L. (2011). *Engineering a Compiler* (2nd ed.). San Francisco, CA: Morgan Kaufmann.

Table 4: Complete SLR(1) ACTION and GOTO Table for Language Grammar G

State	ACTION Table (Terminals)									GOTO Table (Non-Terminals)					
	`let`	`print`	`id`	`num`	`=`	`;`	`+`	`*`	`$`	Program	StmtList	Stmt	Expr	Term	Factor
0	s_2	s_5								4	3	1			
1	r_3	r_3							r_3						
2			s_6												
3	s_2	s_5							r_1			7			
4									acc						
5			s_{10}	s_8									11	12	9
6					s_{13}										
7	r_2	r_2							r_2						
8						r_{11}	r_{11}	r_{11}							
9						r_9	r_9	r_9							
10						r_{10}	r_{10}	r_{10}							
11						s_{14}	s_{15}								
12						r_7	r_7	s_{16}							
13			s_{10}	s_8									17	12	9
14	r_5	r_5							r_5						
15			s_{10}	s_8										18	9
16			s_{10}	s_8											19
17						s_{20}	s_{15}								
18						r_6	r_6	s_{16}							
19						r_8	r_8	r_8							
20	r_4	r_4							r_4						

Table 5: Stepwise Shift-Reduce Parse Trace for `let x = 5 + 10 * y ; print x ;`

Step	State Stack	Input Buffer	Parser Action
1	`0`	`let x = 5 + 10 * y ; print x ; $`	Shift to state 2
2	`0 let 2`	`x = 5 + 10 * y ; print x ; $`	Shift to state 6
3	`0 let 2 id 6`	`= 5 + 10 * y ; print x ; $`	Shift to state 13
4	`0 let 2 id 6 = 13`	`5 + 10 * y ; print x ; $`	Shift to state 8
5	`0 let 2 id 6 = 13 num 8`	`+ 10 * y ; print x ; $`	Reduce 11: Factor → num
6	`0 let 2 id 6 = 13 Factor 9`	`+ 10 * y ; print x ; $`	Reduce 9: Term → Factor
7	`0 let 2 id 6 = 13 Term 12`	`+ 10 * y ; print x ; $`	Reduce 7: Expr → Term
8	`0 let 2 id 6 = 13 Expr 17`	`+ 10 * y ; print x ; $`	Shift to state 15
9	`0 let 2 id 6 = 13 Expr 17 + 15`	`10 * y ; print x ; $`	Shift to state 8
10	`... Expr 17 + 15 num 8`	`* y ; print x ; $`	Reduce 11: Factor → num
11	`... Expr 17 + 15 Factor 9`	`* y ; print x ; $`	Reduce 9: Term → Factor
12	`... Expr 17 + 15 Term 18`	`* y ; print x ; $`	Shift to state 16 (Preserves $*$)
13	`... Term 18 * 16`	`y ; print x ; $`	Shift to state 10
14	`... Term 18 * 16 id 10`	`; print x ; $`	Reduce 10: Factor → id
15	`... Term 18 * 16 Factor 19`	`; print x ; $`	Reduce 8: Term → Term * Factor
16	`... Expr 17 + 15 Term 18`	`; print x ; $`	Reduce 6: Expr → Expr + Term
17	`0 let 2 id 6 = 13 Expr 17`	`; print x ; $`	Shift to state 20
18	`0 let 2 id 6 = 13 Expr 17 ; 20`	`print x ; $`	Reduce 4: Stmt → let id = Expr ;
19	`0 Stmt 1`	`print x ; $`	Reduce 3: StmtList → Stmt
20	`0 StmtList 3`	`print x ; $`	Shift to state 5
21	`0 StmtList 3 print 5`	`x ; $`	Shift to state 10
22	`0 StmtList 3 print 5 id 10`	`; $`	Reduce 10: Factor → id
23	`... print 5 Factor 9`	`; $`	Reduce 9: Term → Factor
24	`... print 5 Term 12`	`; $`	Reduce 7: Expr → Term
25	`... print 5 Expr 11`	`; $`	Shift to state 14
26	`... print 5 Expr 11 ; 14`	`$`	Reduce 5: Stmt → print Expr ;
27	`0 StmtList 3 Stmt 7`	`$`	Reduce 2: StmtList → StmtList Stmt
28	`0 StmtList 3`	`$`	Reduce 1: Program → StmtList
29	`0 Program 4`	`$`	**Accept!**